10

PART 3: PRACTICE AND IMPLICATIONS

The Future Engineer

Three engineers on the Riverside Clinics team open the same finished specification: the Schedule Appointment use case from Chapter 3, the domain model from Chapter 4, the Spring Boot skill from Chapter 7. Each points an AI agent at it separately. Their team lead expects three nearly identical pull requests back within the hour. A use case this precise, she assumes, should produce one obvious answer.

It doesn't. All three implementations pass every test the use case specifies. None breaks a business rule. And no two of them handle a patient trying to book an already-taken slot the same way. One locks the slot in the database the instant a patient starts confirming it. Another lets several patients see the same open slot and resolves the conflict only when someone commits. The third holds the slot for 90 seconds while the patient finishes confirming, then releases it if they don't. The lead's first instinct is that something is wrong. Nothing is wrong. The specification did exactly what it was written to do, and there was still a decision left for a person to make.

Automation and Professional Anxiety

The question the Riverside engineers are really answering, without quite phrasing it this way, is one software has asked before. When compilers replaced hand-written assembly, people asked whether programmers would still be needed. When fourth-generation languages promised that analysts could generate applications directly from specifications, the same question came back under a new name. Low-code platforms revived it again later: would business users simply build the software themselves, no engineer required?

None of those transitions removed the engineer. Each one moved what engineers spent their day doing, away from whatever the new abstraction had just made cheap and toward whatever stayed scarce once it had. That's a pattern repeated across decades of software history, not a guarantee about what happens next. AI-assisted implementation goes further than any of the three before it, though. It doesn't just remove one layer of manual translation. Handed a finished specification, it can produce a working, tested implementation directly. What matters is what it still leaves for an engineer to do, and the scene at the start of this chapter already contains the answer.

What Automation Made Cheap

Call the first half of the old job **syntax fluency**: knowing the keywords, the framework calls, the correct way to wire a repository to a service to a controller. For most of software's history, syntax fluency was the scarce half. An organization paid for people who could reliably produce it, and reliably producing it was most of what separated a senior engineer from a beginner.

AI agents made syntax fluency cheap. Given the Schedule Appointment specification, an agent produces syntactically correct, idiomatic, tested code for it in minutes, in whichever stack the project's skill tells it to use. What an agent cannot supply is the other half of the job, **judgment**: knowing which of several syntactically correct implementations serves the business, and being able to say why. The three engineers at Riverside all had syntax fluency. AI made that part cheap for all of them. What separated their pull requests was judgment, and judgment is exactly what stayed scarce.

Judgment, in turn, draws on something specific: knowing the business well enough to notice when a specification's silence matters. An engineer who has spent two years on Riverside's scheduling system knows that clinic operations cares more about a doctor's calendar never double-booking than about shaving a few hundred milliseconds off a booking request. A generically skilled engineer, however fast at producing syntax, has to ask before they know that, or guess. Domain knowledge always mattered more than syntax speed. The margin just got wider.

> **Syntax got cheap. Judgment didn't.**

Judgment Work That Remains

Judgment isn't the only thing this abstraction doesn't reach. Four capabilities in particular don't show up in a specification, however well it's written, because a specification records what they produce, not the work of producing it.

Negotiation is one: getting the clinic's operations team and its front desk staff to agree how much notice a cancellation needs, before that agreement becomes the 24-hour rule in Chapter 3's use case. Ethics is another: deciding what the system should do when the technically correct answer harms a patient in a way no test case anticipated. A third is leadership, getting three engineers with three defensible implementations to agree on which one ships, then getting the team to move forward once that decision is made. The fourth is specific domain expertise: knowing, because you've sat in the room, that clinic operations chooses predictability over speed, before anyone has written it down.

None of the three engineers' pull requests needed any of that to be syntactically correct. All three needed it to be chosen.

Reviewing an Underspecified Decision

Once syntax is cheap, review work moves with it. Checking whether generated code matches the codebase's formatting conventions used to take time; that check is worth almost nothing today, since an agent following the project's skill gets it right by default. What review is for now is the question none of that touches: does this implementation still mean what the specification said, and where it diverges from another correct version, was that an oversight or a deliberate design choice?

1. Confirm the implementation satisfies the specification's business rules and acceptance criteria. A passing test suite checks this; a reviewer's own reading of the use case against the code confirms it.
2. Find exactly where two spec-compliant implementations diverge. That divergence marks a place the specification was silent by design, an implementation detail, not a place it was violated.
3. For each divergence, name what it affects. Correctness is already settled at step one, so what's left is behavior under load, the patient's experience, or how hard the code is to change later.
4. Weigh those effects against something the specification doesn't state, because it isn't a business rule: expected traffic, the team's operational maturity, where the business plans to take the feature next.
5. Choose, and write down why, in an architecture decision record or a comment on the pull request, so the next engineer, or the next AI run, inherits the reasoning and not just the code.

Worked Example: Three Ways to Hold a Slot

Return to the three pull requests at the start of this chapter. Each implements the Schedule Appointment use case's core rule correctly: one patient, one eligible doctor, one open slot, inside the 90-day booking window agreed with clinic operations back in Chapter 3. Laid side by side, though, they make different bets about a question the use case leaves open, because answering it isn't a business rule.

APPROACH	WHEN TWO PATIENTS REACH FOR ONE SLOT	WHERE IT WINS	WHAT IT COSTS
Lock at confirm	Database row lock held from the moment confirmation starts	Simple to reason about; the winning patient loses no work	The second patient stalls waiting on the lock, worse under high volume
Resolve at commit	Both patients proceed until commit; the loser sees "slot no longer available"	Scales well under contention; nobody waits	The losing patient's confirmation work is wasted and needs its own retry screen
90-second hold	First patient gets an exclusive hold while the confirmation is in progress	Matches how patients already expect travel booking to feel	Needs a background job to expire stale holds, more moving parts to operate

The specification doesn't say which of the three is correct. It shouldn't. Deciding between them needs information the specification correctly leaves out: how often two Riverside patients collide on the same slot, whether clinic operations wants the reassurance of a visible hold, whether the team already operates the kind of background job the third option needs. That decision belongs to the lead, and it's engineering judgment, not a gap in the specification or a flaw in what the AI understood.

What follows is speculation. If the valuable half of the job is shifting from producing syntax to organizing and validating knowledge, a curriculum built heavily around requirements engineering, domain modeling, and what Chapter 6 called context engineering is one reasonable way engineering education could respond, and a title like "knowledge engineer" is one name someone might give the resulting role.

Limitations

▲ **WARNING**

Judgment being scarce doesn't make it automatic. A reviewer who waves through three implementations because a human chose one has skipped the judgment call, the same way a rubber-stamped syntax review used to skip catching a bug. The bar moved. It didn't disappear.

Treating this shift as license to relax review effort	A team stops asking which implementation the business needs and ships whichever pull request finished first, the exact shortcut this book has spent nine chapters arguing against.
Treating domain expertise as sufficient on its own	An engineer who knows the clinic's rules well starts deciding from memory instead of the specification, reintroducing the undocumented, in-someone's-head knowledge that made brownfield systems (Chapter 9) hard to recover in the first place.

This chapter is on solid ground describing what has already moved: syntax cheap, judgment scarce, review's attention shifting toward intent. It's on thinner ground guessing at exactly what that produces ten or twenty years out, which is why the curriculum sketch and the job title on the previous page were offered as speculation, with no date attached and no claim that either arrives.

Checklist: Syntax-to-Judgment Self-Assessment

- ☐ If your AI agent were unavailable for a week, could you still produce syntactically correct code for your stack, or would you only recognize whether the agent's output was correct?
- ☐ When two AI-generated implementations of the same use case both pass every test, can you name the trade-off that should decide between them for your team?
- ☐ Do you know the business reason behind a specific field, status value, or edge case in your system without asking someone else first?
- ☐ In your last three code reviews, were most of your comments about formatting and style, or about whether the code matched the specification's intent?
- ☐ Could you write the specification for a feature you maintain regularly, from scratch, without an existing one to copy?

Landing on the syntax side of most of these questions points at what's worth building next: domain knowledge and specification judgment, not more syntax practice.

Summary

- Every prior abstraction, compilers, fourth-generation languages, low-code, raised the same fear about engineers disappearing. The engineers stayed. Each shift changed what they spent their time on.
- AI made syntax fluency cheap. It didn't make judgment, the ability to choose well between several correct implementations, any less scarce.
- Domain knowledge now outweighs syntax speed by a wider margin than it used to, because the syntax half of the job got automated and the judgment half didn't.
- A specification can be entirely correct and still leave more than one valid implementation standing. Choosing between them, and explaining why, is still engineering work.

DISCUSSION QUESTIONS

- When someone on your team chooses between two correct implementations, where does that reasoning get written down, and who reads it later?
- If two engineers on your team produced different, both-correct implementations of the same use case tomorrow, who would be equipped to choose between them, and why?

PRACTICAL EXERCISE

Pull the last pull request your team accepted without much discussion, one that came back from an AI agent fast and clean, and ask what would have been different about it if two other engineers had implemented the same specification separately. If you can't answer, that's the skill to build next.

The Future Engineer ends here.